



Plotnine

```
posit::conf(2026)
```

- Some examples
- Why visualize?
- Plotnine features
- Plotnine concepts
- Resources for working with Plotnine
- Bonus: Great Tables



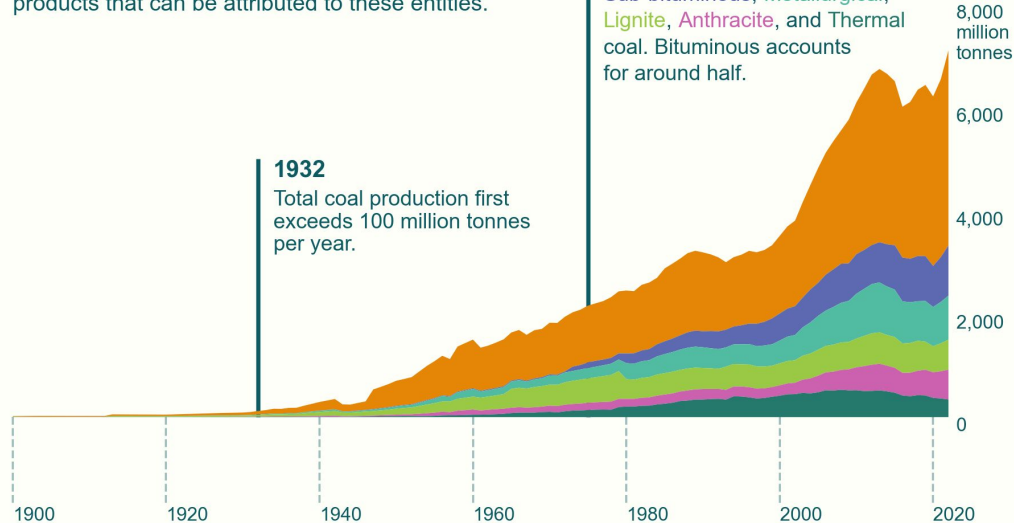
Some examples

Coal production since 1900

Carbon Majors is a database of historical production data from 122 of the world's largest oil, gas, coal, and cement producers. This data is used to quantify the direct operational emissions and emissions from the combustion of marketed products that can be attributed to these entities.

Coal types

Total coal production includes production of **Bituminous**, **Sub-bituminous**, **Metallurgical**, **Lignite**, **Anthracite**, and Thermal coal. Bituminous accounts for around half.



Data: Carbon Majors

Graphic: Nicola Rennie (@nrennie)

Stacked area chart by Nicola Rennie

The rise of artificial intelligence over the last 8 decades: As training computation has increased, AI systems have become more powerful

Computation is measured in floating point operators (FLOP). One FLOP is equivalent to one addition, subtraction, multiplication, or division of two decimals. The data is shown on a logarithmic scale: each grid line is a 100-fold increase in computation.

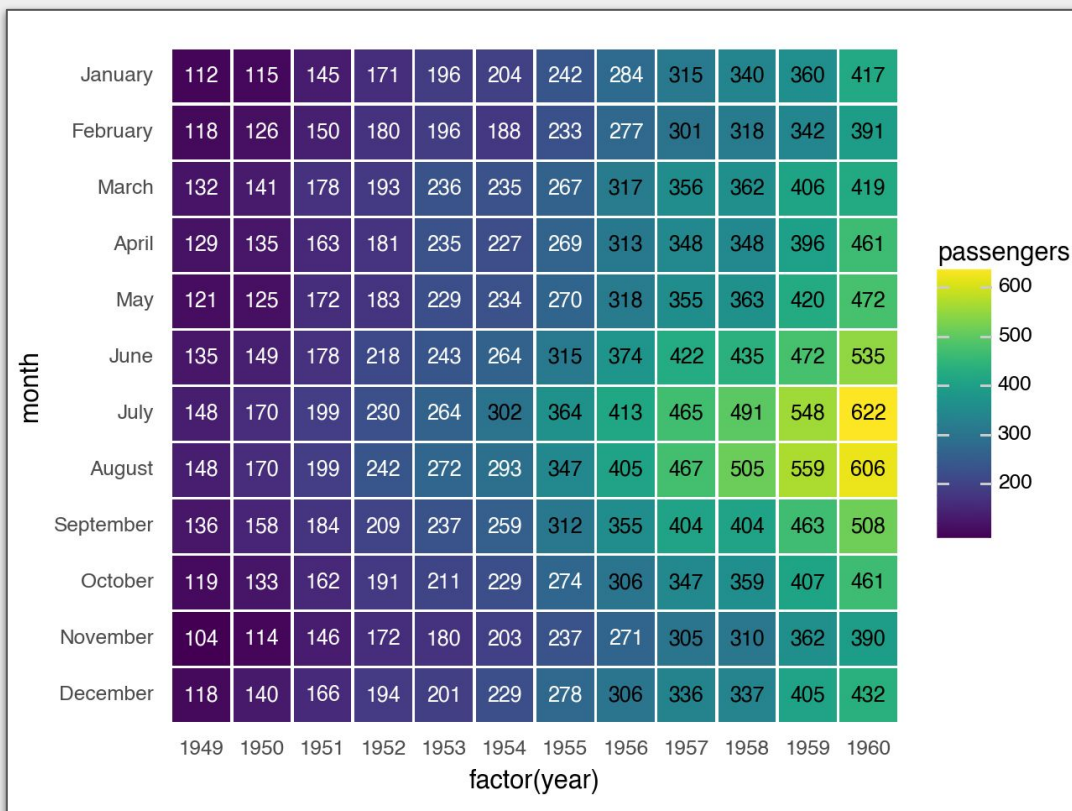
The color indicates the domain of the AI system: Vision (red), Games (green), Drawing (blue), Language (purple), Other (pink).

Training computation that was used to train the AI systems.

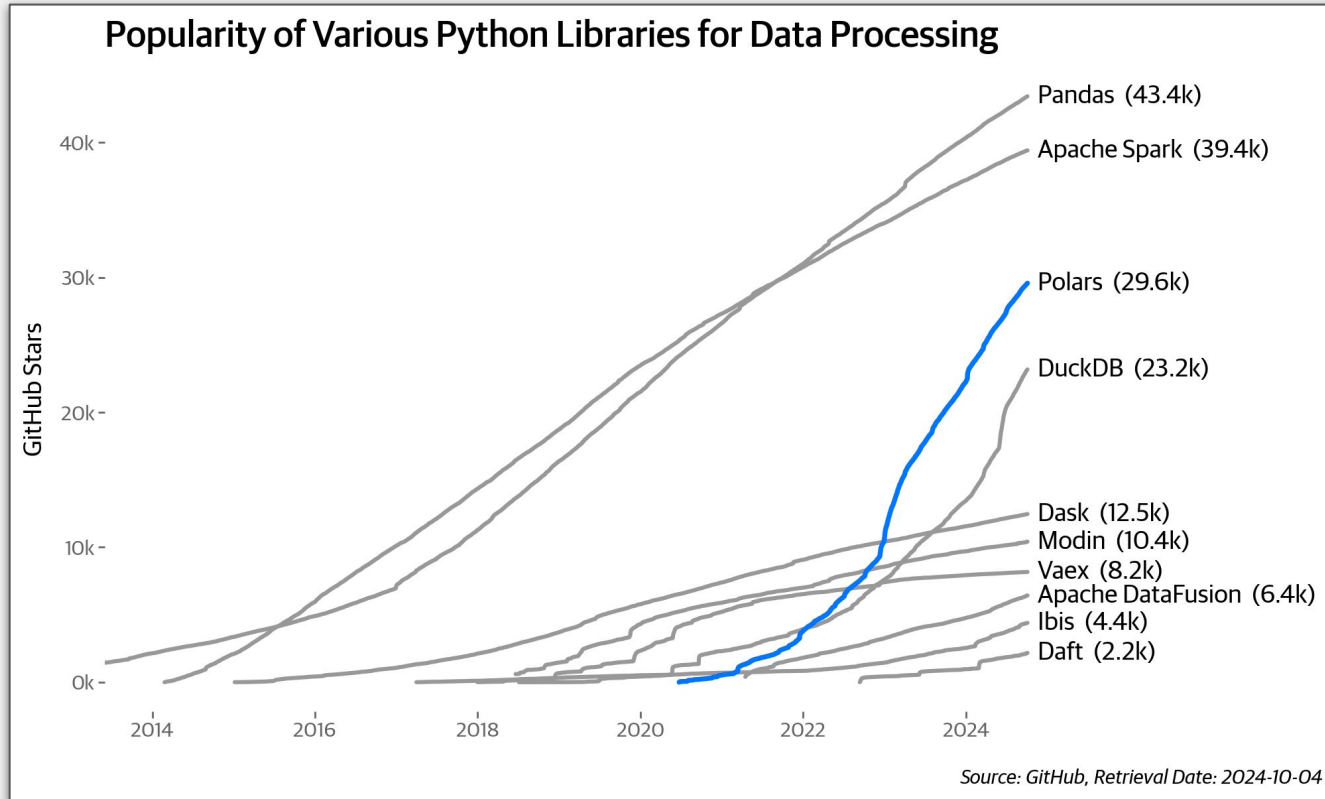


Data: Sevilla et al. (2022) | Visual: Greg Chism | Original: OurWorldInData.org - Charlie Giattino, Edouard Mathieu, and Max Roser
Licensed under CC-BY 4.0 International

Annotated scatter plot by Greg Chism

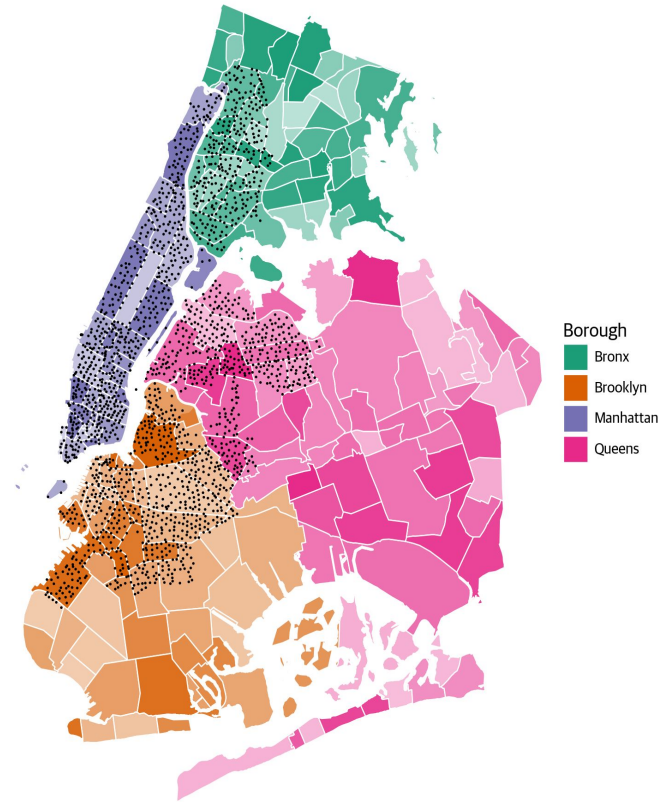


Heatmap from Plotnine's documentation



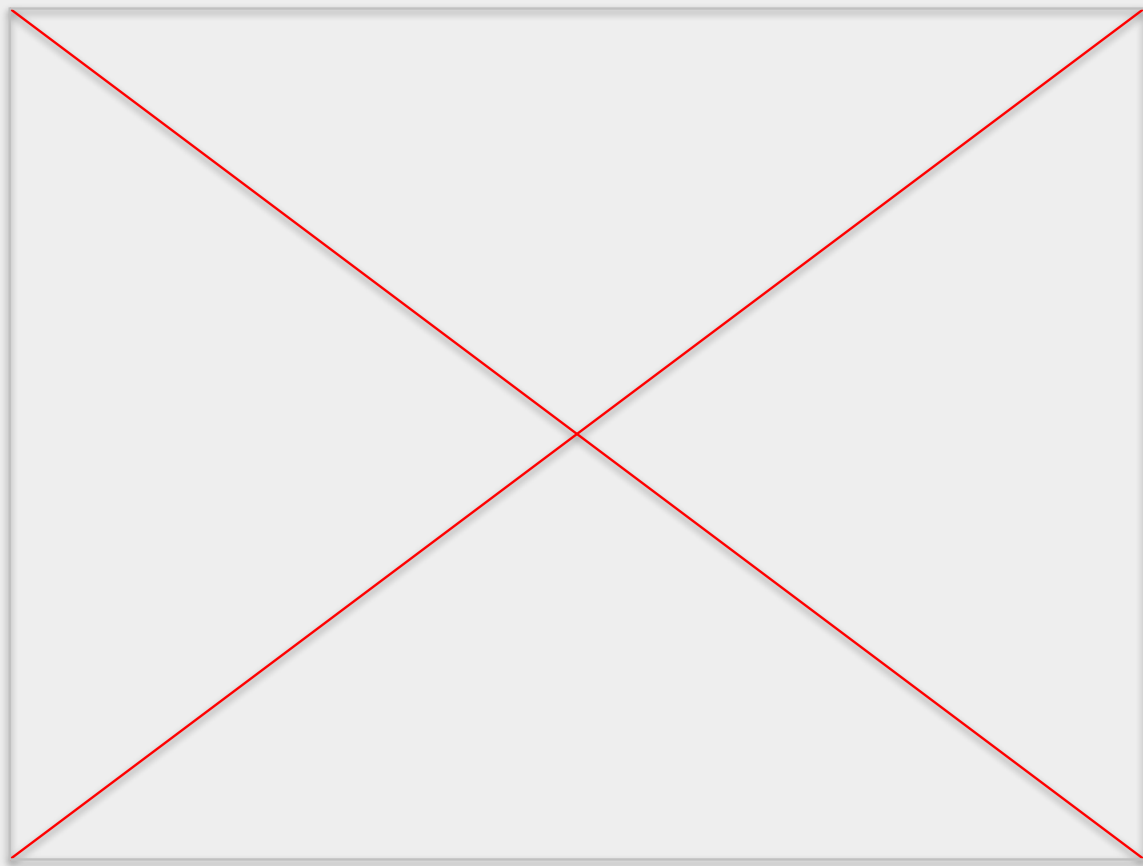
Line graph by Jeroen Janssens

New York City Neighborhoods and Citibike Stations
2143 stations across 106 neighborhoods



Source: <https://citibikenyc.com/system-data>

*NYC Citi Bike stations
by Jeroen Janssens*



Dimensionality Reduction, Animated by Jeroen Janssens

Why visualize?

Anscombe's quartet

anscombe_quartet

shape: (44, 3)

dataset	x	y
str	f64	f64
"I"	10.0	8.04
"I"	8.0	6.95
"I"	13.0	7.58
"I"	9.0	8.81
"I"	11.0	8.33
...	...	...
"IV"	8.0	5.25
"IV"	19.0	12.5
"IV"	8.0	5.56
"IV"	8.0	7.91
"IV"	8.0	6.89

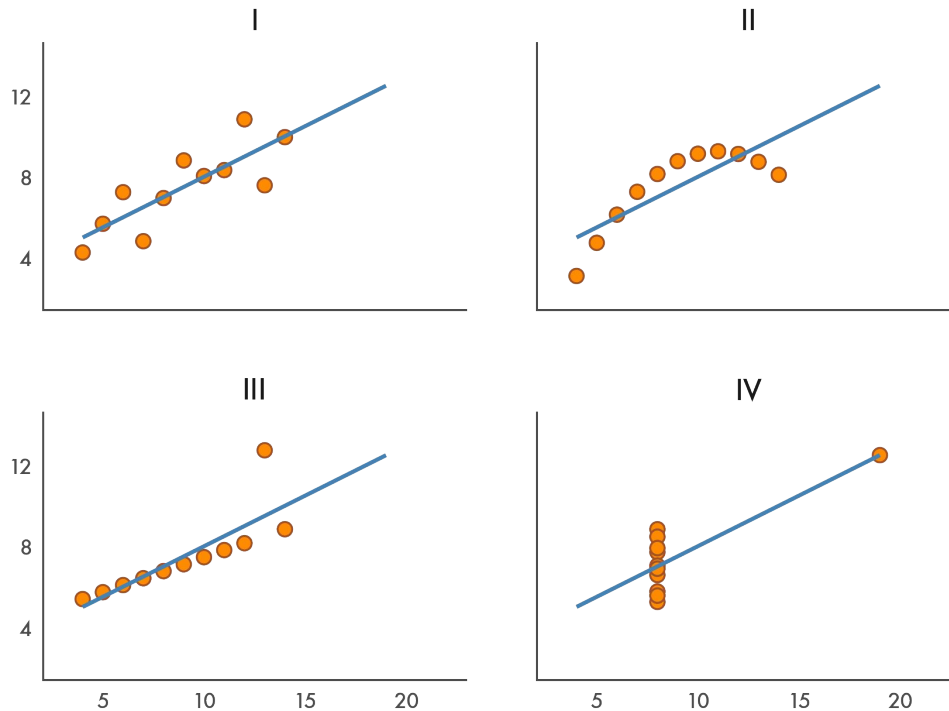
```
anscombe.groupby("dataset").agg(  
    x_mean=pl.col("x").mean(),  
    y_mean=pl.col("y").mean(),  
    x_std=pl.col("x").std(),  
    y_std=pl.col("y").std(),  
    corr=pl.corr("x", "y"),  
)
```

shape: (4, 6)

dataset	x_mean	y_mean	x_std	y_std	corr
str	f64	f64	f64	f64	f64
"III"	9.0	7.5	3.316625	2.030424	0.816287
"II"	9.0	7.500909	3.316625	2.031657	0.816237
"I"	9.0	7.500909	3.316625	2.031568	0.816421
"IV"	9.0	7.500909	3.316625	2.030579	0.816521

Anscombe's quartet

Anscombe's Quartet

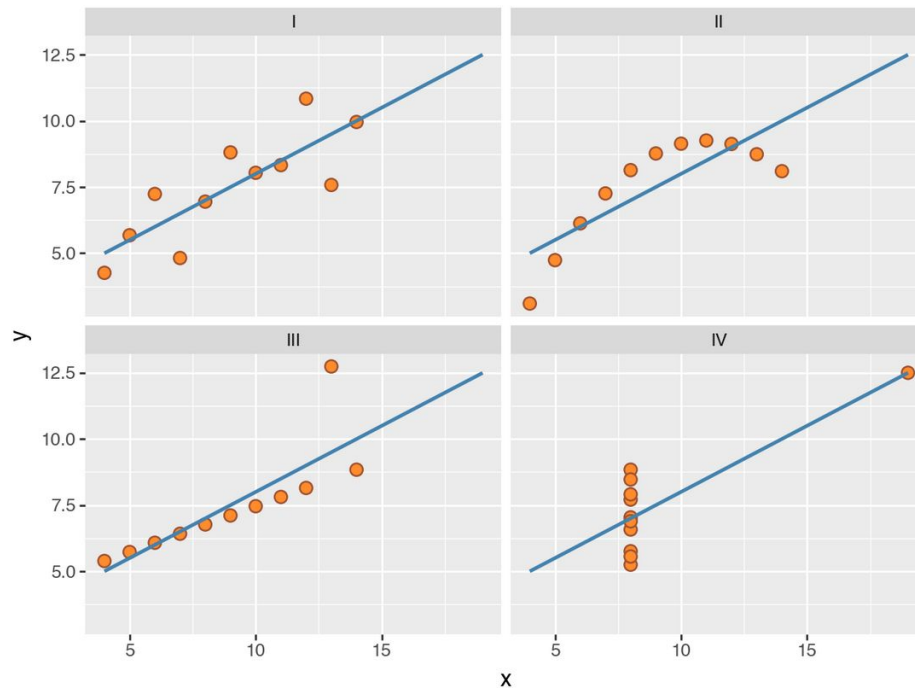


Anscombe's quartet using Plotnine (the whole shebang)

```
(
  ggplot(anscombe, aes("x", "y")) +
  geom_point(color="sienna", fill="darkorange", size=3) +
  geom_smooth(method="lm", se=False, fullrange=True, color="steelblue") +
  scale_y_continuous(breaks=(4, 8, 12)) +
  coord_equal(xlim=(3, 22), ylim=(2, 14)) +
  facet_wrap("dataset") +
  labs(title="Anscombe's Quartet") +
  theme_tufte(base_family="Futura") +
  theme(
    axis_line=element_line(color="#4d4d4d"),
    axis_ticks_major=element_line(color="#00000000"),
    axis_title=element_blank(),
    dpi=144,
    panel_spacing=0.066,
    strip_text=element_text(size=12, weight=700),
    title=element_text(size=16, margin={"b": 20})
  )
)
```

Anscombe's quartet using Plotnine (minimum viable plot)

```
(  
  ggplot(anscombe, aes("x", "y")) +  
  geom_point(fill="darkorange", color="sienna", size=3) +  
  geom_smooth(method="lm", se=False, fullrange=True, color="steelblue") +  
  facet_wrap("dataset")  
)
```



Plotnine

Plotnine in a nutshell

- Created by Hassan Kibirige
- Based on ggplot2 by Hadley Wickham
 - Plotnine is stable, but not as complete
 - Subtle syntax differences because Python and R
 - Still, consistent syntax
 - We can use ggplot2's extensive documentation
- Under active development, supported by Posit
- Documentation at <https://plotnine.readthedocs.io/>

Plotnine Concepts

1. Tidy data
2. Mapping columns to aesthetics
3. Optional statistical transformation
4. From data values to aesthetic values using scales
5. Layers and inheritance
6. Dynamic and static aesthetics

Long is good:

```
anscombe_quartet
```

```
shape: (44, 3)
```

dataset	x	y
str	f64	f64
"I"	10.0	8.04
"I"	8.0	6.95
"I"	13.0	7.58
"I"	9.0	8.81
"I"	11.0	8.33
...	...	...
"IV"	8.0	5.25
"IV"	19.0	12.5
"IV"	8.0	5.56
"IV"	8.0	7.91
"IV"	8.0	6.89

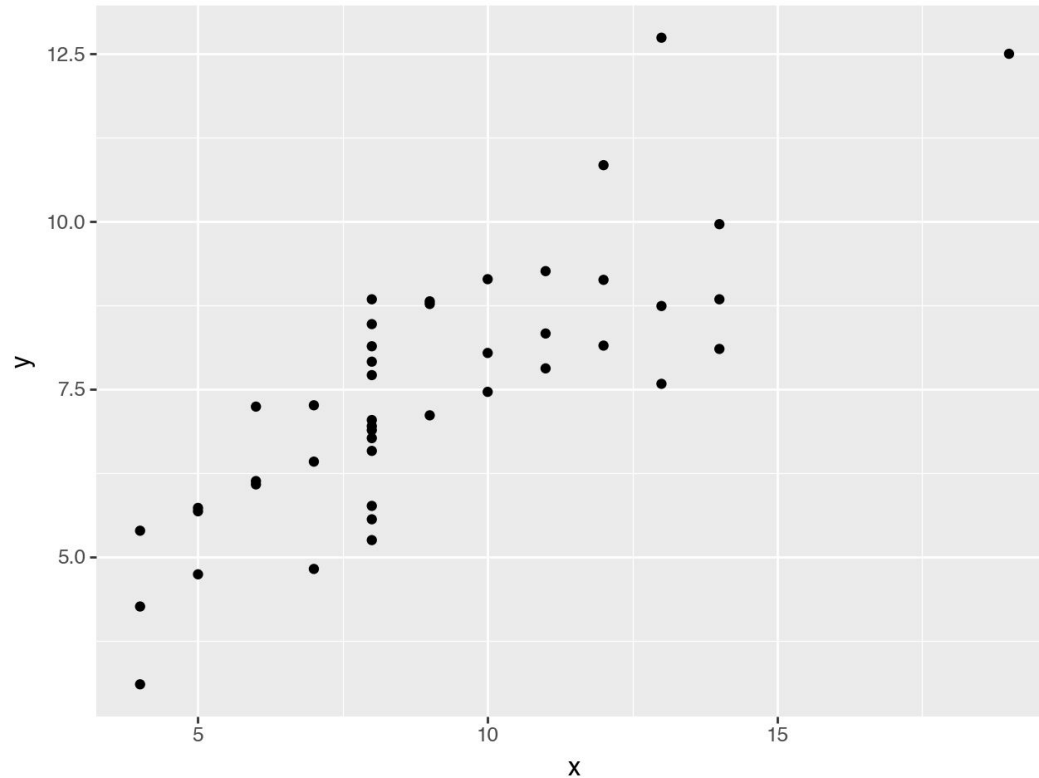
Wide is bad:

```
shape: (11, 9)
```

index	x_I	x_II	x_III	x_IV	y_I	y_II	y_III	y_IV
i64	f64	f64	f64	f64	f64	f64	f64	f64
0	10.0	10.0	10.0	8.0	8.04	9.14	7.46	6.58
1	8.0	8.0	8.0	8.0	6.95	8.14	6.77	5.76
2	13.0	13.0	13.0	8.0	7.58	8.74	12.74	7.71
3	9.0	9.0	9.0	8.0	8.81	8.77	7.11	8.84
4	11.0	11.0	11.0	8.0	8.33	9.26	7.81	8.47
...	...	...	...	...	...	...	...	...
6	6.0	6.0	6.0	8.0	7.24	6.13	6.08	5.25
7	4.0	4.0	4.0	19.0	4.26	3.1	5.39	12.5
8	12.0	12.0	12.0	8.0	10.84	9.13	8.15	5.56
9	7.0	7.0	7.0	8.0	4.82	7.26	6.42	7.91
10	5.0	5.0	5.0	8.0	5.68	4.74	5.73	6.89

Plotnine concept 2: Mapping columns to aesthetics

```
: ggplot(anscombe_quartet, aes("x", "y")) + geom_point()
```



Resources for working with Plotnine

Components of a Plotnine visualization and function hierarchy

- Default dataset (`p1.DataFrame`)
- One or more layers:
 - Aesthetic mappings (`p9.aes`)
 - Geometric object (`p9.geom_*`)
 - Statistical transformation (`p9.stat_*`)
- Scale for each aesthetic mapping (`p9.scale_*`)
- Coordinate system (`p9.coord_*`)
- Facet specification (`p9.facet_*`)
- Labels (`p9.labs`)
- Theme (`p9.theme`)

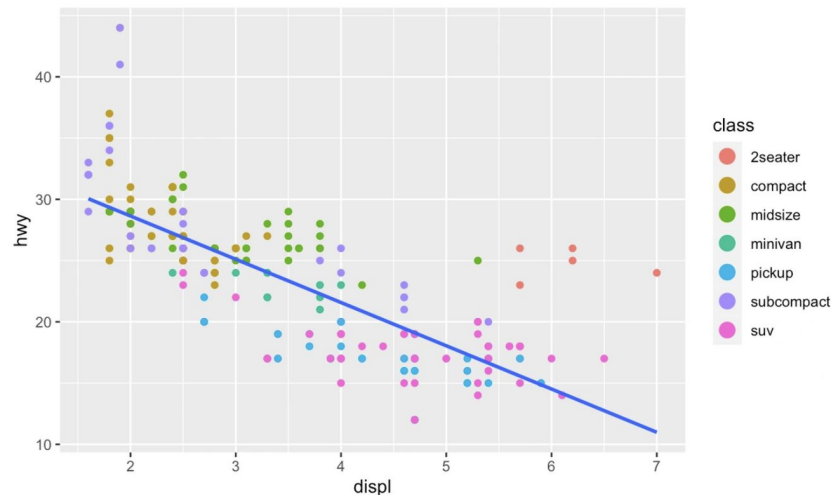
Plotnine Cheat Sheet



Heuristics for Translating Ggplot2 Code to Plotnine Code

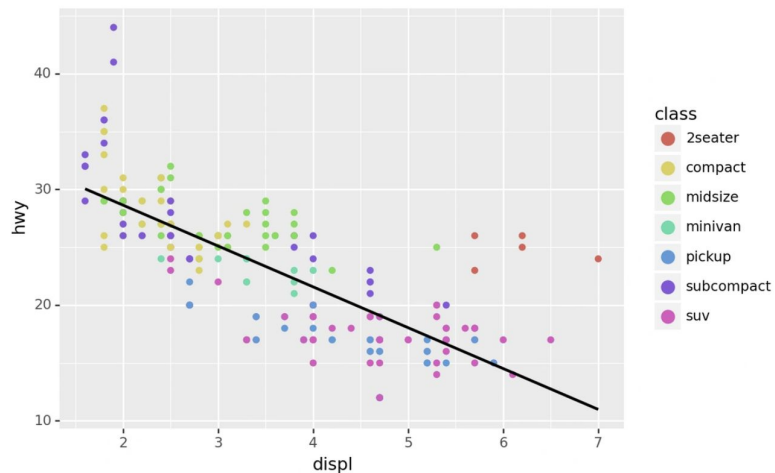
```
library(ggplot2)

ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(colour = class)) +
  geom_smooth(se = FALSE, method = "lm") +
  guides(colour = guide_legend(override.aes = list(size = 4)))
```



Heuristics for Translating Ggplot2 Code to Plotnine Code

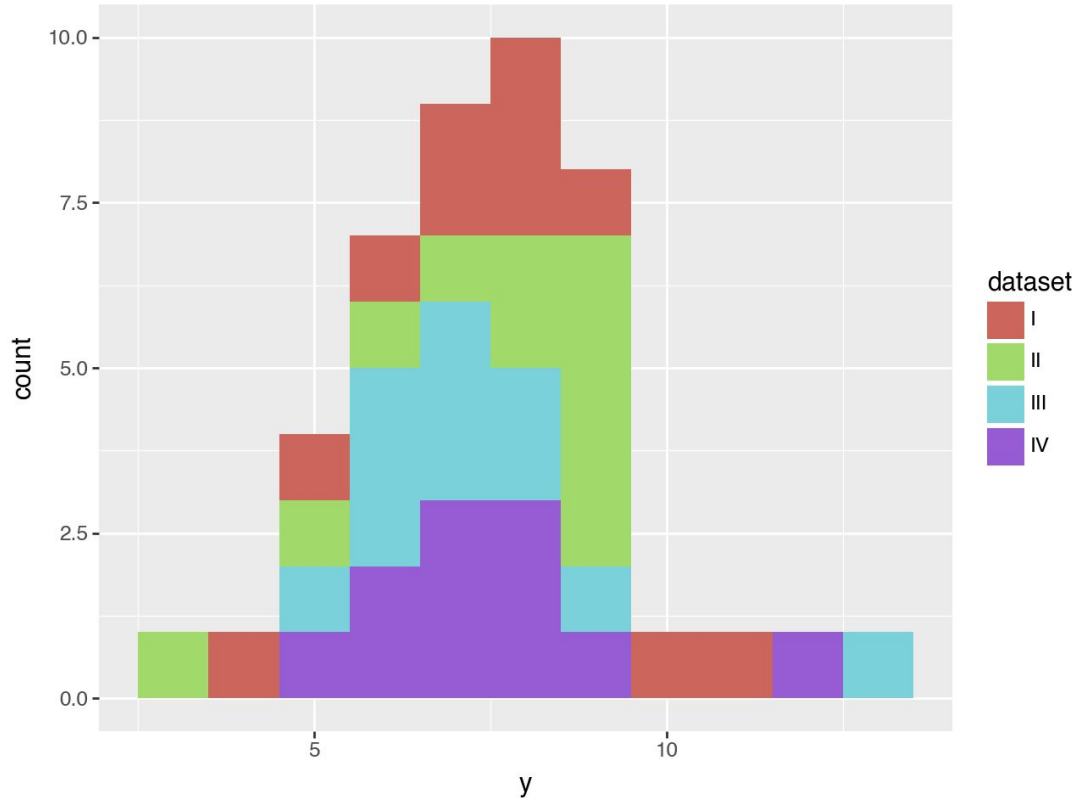
```
from plotnine import *  
from plotnine.data import mpg  
  
ggplot(mpg, aes("displ", "hwy")) +\  
  geom_point(aes(colour="class")) +\  
  geom_smooth(se=False, method="lm") +\  
  guides(colour=guide_legend(override_aes={"size": 4}))
```



Exercise 1

Plotnine concept 3: Optional statistical transformation

```
ggplot(anscombe_quartet, aes("y", fill="dataset")) + geom_histogram(binwidth=1)
```



Plotnine concept 3: Optional statistical transformation

1. **geom_bar()** begins with the **diamonds** data set

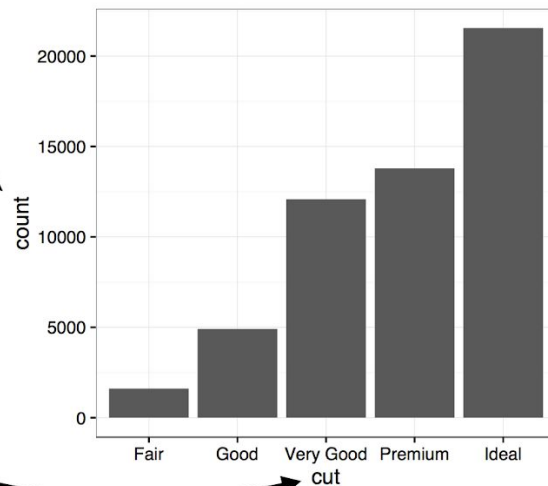
carat	cut	color	clarity	depth	table	price	x	y	z
0.23	Ideal	E	SI2	61.5	55	326	3.95	3.98	2.43
0.21	Premium	E	SI1	59.8	61	326	3.89	3.84	2.31
0.23	Good	E	VS1	56.9	65	327	4.05	4.07	2.31
0.29	Premium	I	VS2	62.4	58	334	4.20	4.23	2.63
0.31	Good	J	SI2	63.3	58	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...

stat_count()

2. **geom_bar()** transforms the data with the "count" stat, which returns a data set of cut values and counts.

cut	count	prop
Fair	1610	1
Good	4906	1
Very Good	12082	1
Premium	13791	1
Ideal	21551	1

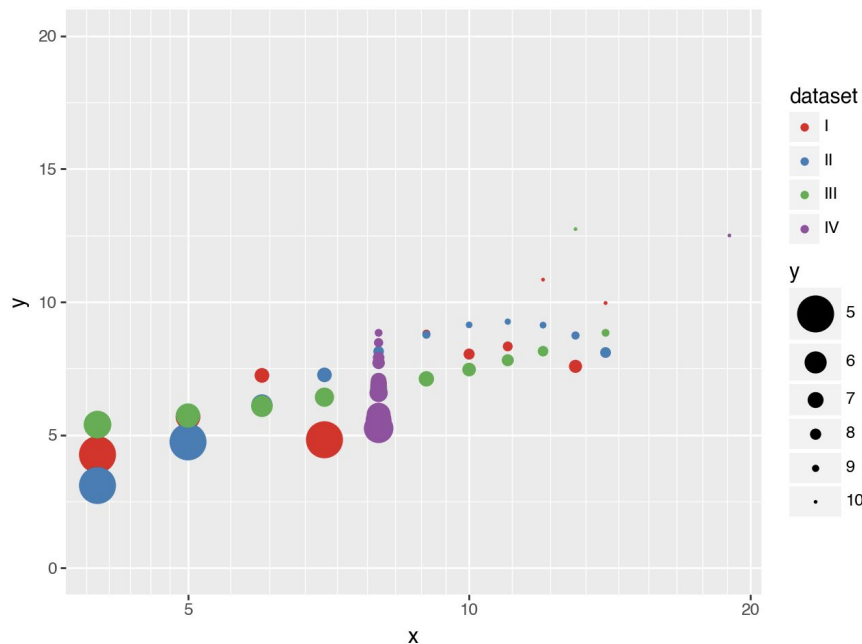
3. **geom_bar()** uses the transformed data to build the plot. cut is mapped to the x axis, count is mapped to the y axis.



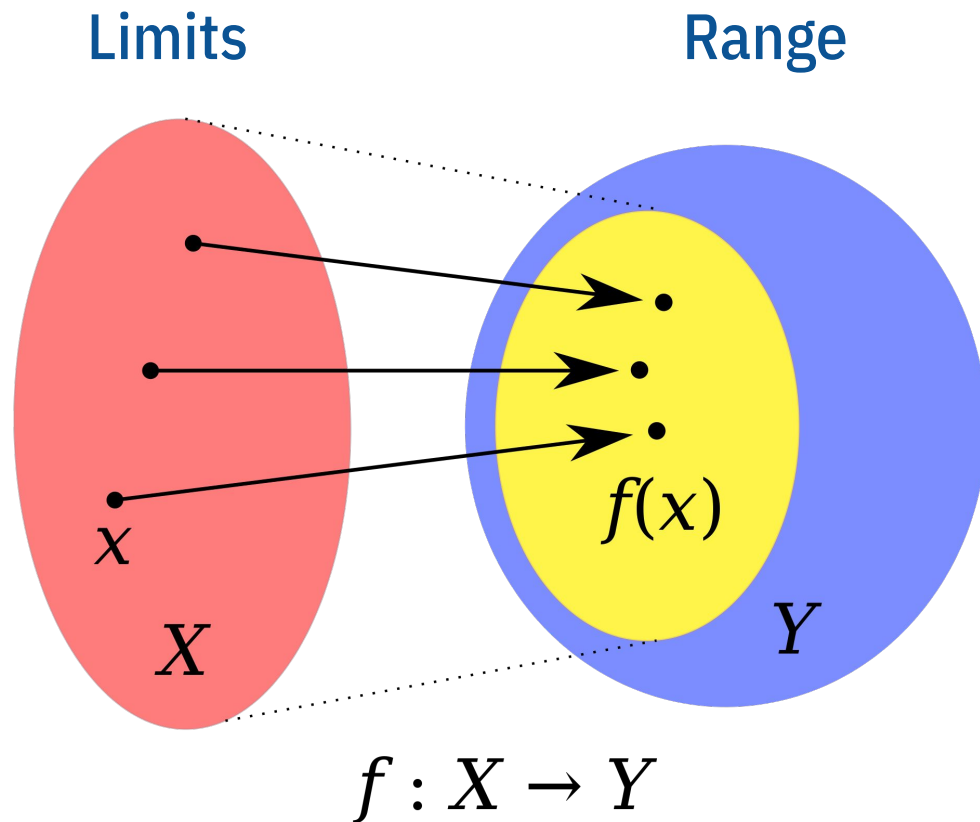
Exercise 2

Plotnine concept 4: From data values to aesthetic values using scales

```
(  
  ggplot(anscombe_quartet, aes("x", "y", color="dataset", size="y"))  
  + geom_point()  
  + scale_x_log10()  
  + scale_y_continuous(limits=(0, 20))  
  + scale_color_brewer(type="qual", palette="Set1")  
  + scale_size_continuous(limits=(5, 10), range=(10, 0.1))  
)
```



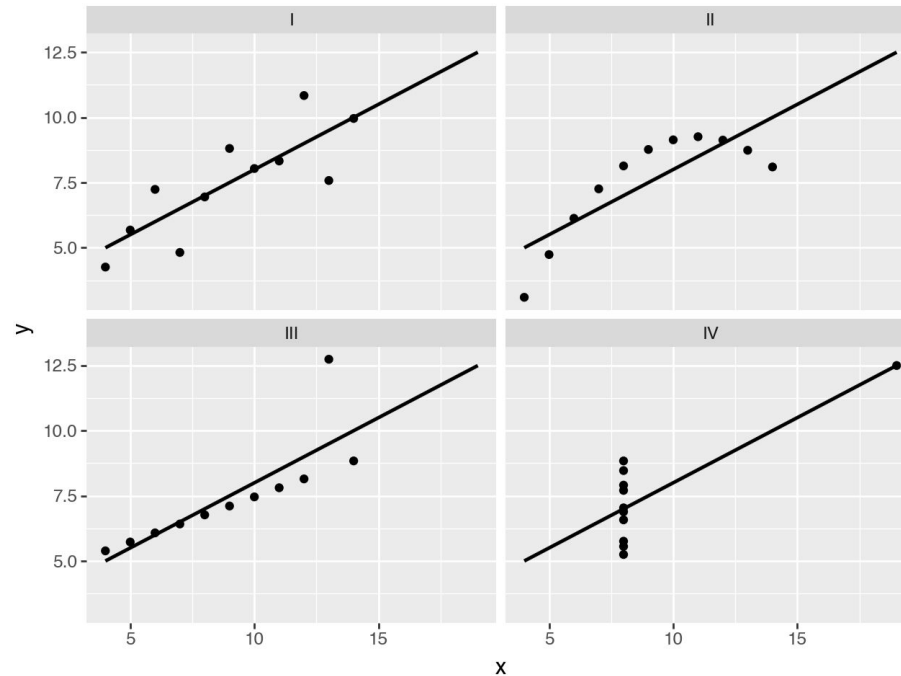
Plotnine concept 4: From data values to aesthetic values using scales



Exercise 3

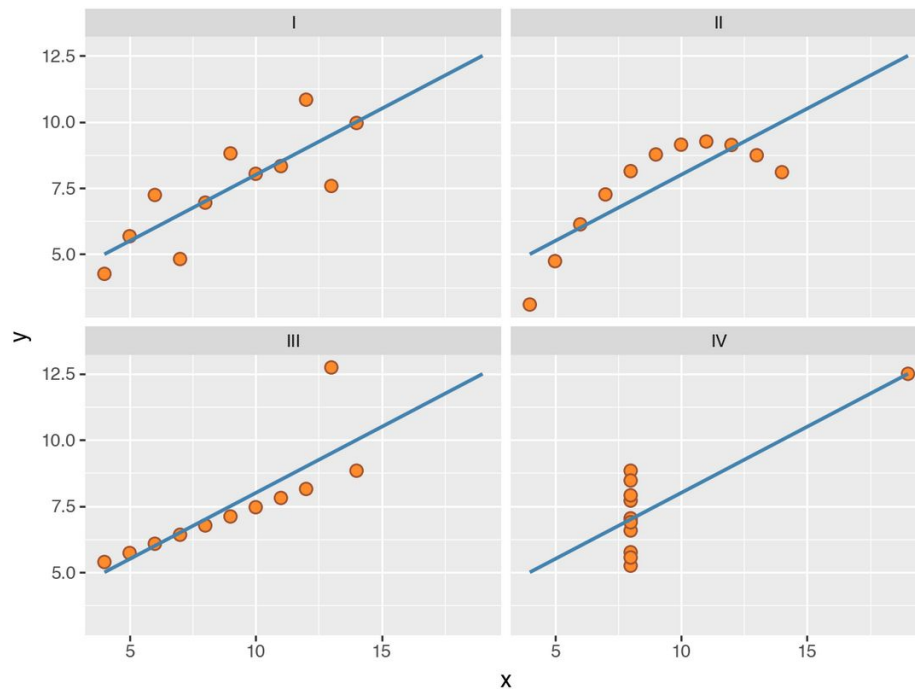
Plotnine concept 5: Layers and inheritance

```
(  
  ggplot(anscombe_quartet, aes("x", "y")) +  
  geom_point() +  
  geom_smooth(method="lm", se=False, fullrange=True) +  
  facet_wrap("dataset")  
)
```



Plotnine concept 6: Dynamic and static aesthetics

```
(  
  ggplot(anscombe, aes("x", "y")) +  
    geom_point(fill="darkorange", color="sienna", size=3) +  
    geom_smooth(method="lm", se=False, fullrange=True, color="steelblue") +  
    facet_wrap("dataset")  
)
```



Exercise 4

Appendix

Theme elements (replace "." with "_")

ggplot2 Theme Elements

`theme(element_name = element_function())`

- `element_text()`
- `element_line()`
- `element_rect()`
- `element_blank()`

Axis elements:

- `axis.ticks`
`element_line()`
- `axis.title`
`element_text()`
- `axis.text`
`element_text()`
- `axis.line`
`element_line()`

Plot elements:

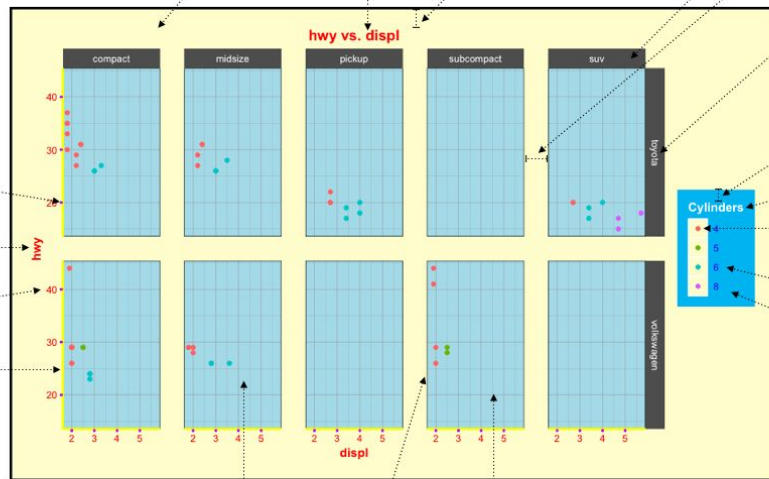
- `plot.background`
`element_rect()`
- `plot.title`
`element_text()`
- `plot.margin`
`margin()`

Facetting elements:

- `strip.background`
`element_rect()`
- `panel.spacing`
`unit()`
- `strip.text`
`element_text()`

Legend elements:

- `legend.margin`
`margin()`
- `legend.title`
`element_text()`
- `legend.key`
`element_rect()`
- `legend.text`
`element_text()`
- `legend.background`
`element_rect()`



`panel.background`
`element_rect()`

`panel.grid`
`element_line()`

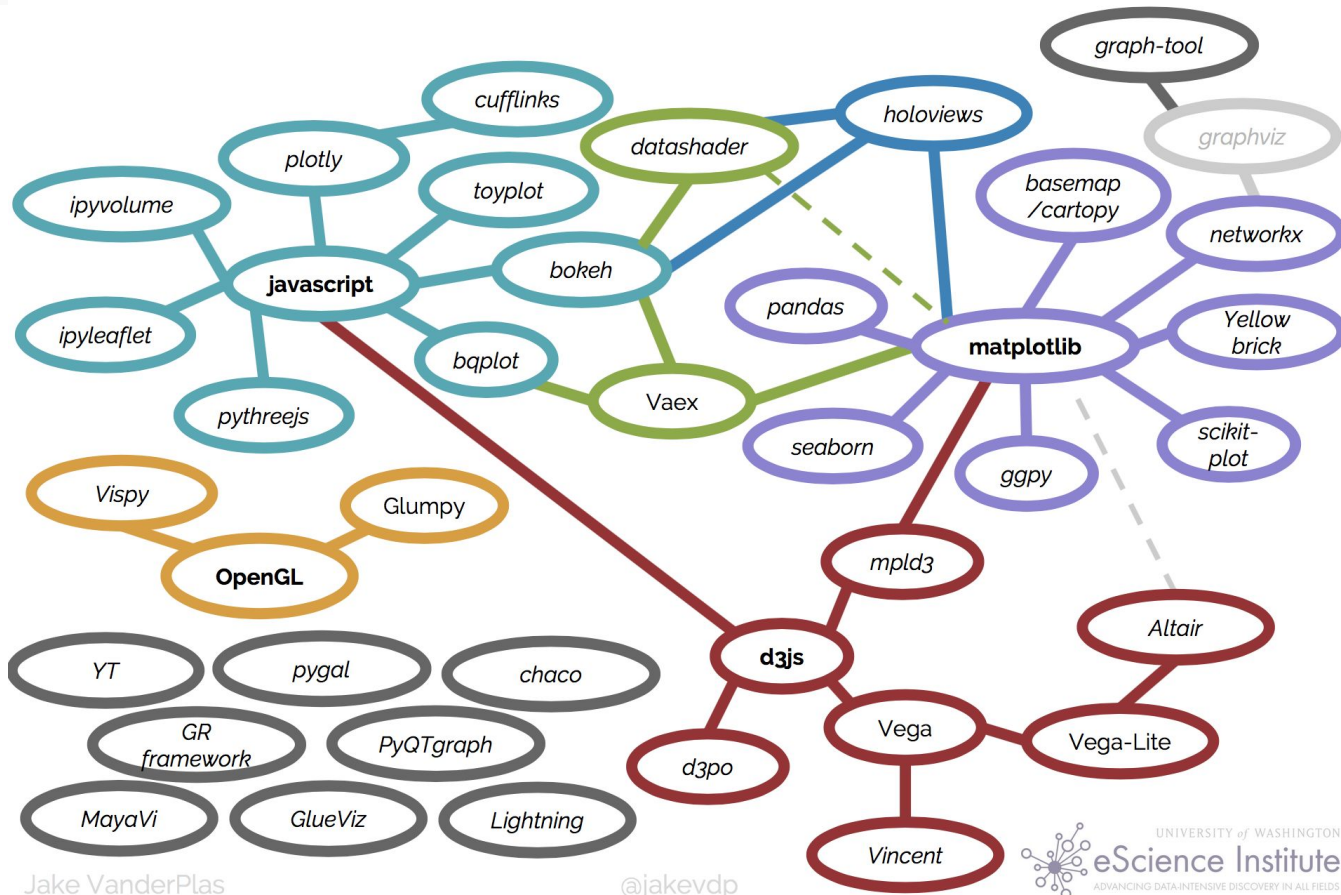
`panel.border`
`element_rect(fill = NA)`

Panel elements:

henrywang.nl

Derived from "ggplot2: Elegant Graphics for Data Analysis"

An incomplete overview of visualization packages in Python



- [Plotnine: Grammar of Graphics for Python | Jeroen Janssens](#)
- [Data visualization with ggplot2 :: CHEAT SHEET](#)
- [Heuristics for Translating Ggplot2 Code to Plotnine Code | Jeroen Janssens](#)
- [A Layered Grammar of Graphics](#)
- [ColorBrewer](#)
- [Choosing Colormaps — Matplotlib 2.0.2 documentation](#)
- [Plotnine](#)